```
</script>
</body>
</html>
```

It starts by defining the HTTP method and URL to the API. In the case of a cross-origin request, the URL is the full URL to the resource. There is no other special information that needs to be set to delineate a request as cross-origin. The browser does the work of parsing the URL and determining if the request is a same-origin or a cross-origin request.

The code then creates a new `XMLHttpRequest` object, and verifies that it supports CORS by checking that the `XMLHttpRequest` object has a `withCredentials` property. The `withCredentials` property will be covered in more detail later in this chapter. For now, all you need to know is that if the `XMLHttpRequest` object has a `withCredentials` property, then it supports CORS.

The `open` method is called to set the values of the HTTP method and URL. The `open` method also does some basic validation to ensure that the HTTP method and URL are valid. For example, if the HTTP method contains a space, calling `open` will throw a syntax error.

ADDING HTTP HEADERS

The `setRequestHeader` method lets you add HTTP headers to the request. This method accepts a header key and header value, and includes the header on the request. The following code snippet, which should be included after the `open` method, shows an example of using the `setRequestHeader` method to set the X-Requested-With request header:

```
xhr.setRequestHeader('X-Requested-With', 'CORS in Action');
```

There are some headers that are set by the browser that can't be set by the user (see the following sidebar for the complete list). As you'll see in the next chapter, the browser sets an Origin header on cross-origin requests. If you try to override this header in your code, the browser will ignore your value. This is a security measure that helps prevent user code from overriding trusted header values. The server can trust these values because it knows the user hasn't accidentally (or maliciously) tainted the value.

> **Setting request headers**
>
> Here is the list of headers that cannot be set by the `setRequestHeader` method: Accept-Charset, Accept-Encoding, Access-Control-Request-Headers, Access-Control-Request-Method, Connection, Content-Length, Cookie, Cookie2, Date, DNT, Expect, Host, Keep-Alive, Origin, Referer, TE, Trailer, Transfer-Encoding, Upgrade, User-Agent, Via, and any headers starting with 'Proxy-' or 'Sec-'.
>
> These headers have special meaning and can only be set by the browser. There is no error if the code tries to set the header. The value is just ignored.

The server has to give its permission for the client to include custom request headers on a cross-origin request. This behavior is unique to cross-origin requests; same-origin